FINAL YEAR PROJECT DOCUMENTATION

Books Library
Bilingual Reading and Audio Application

تقرير مشروع التخرج
لتطبيق مكتبة الكتب ثنائي اللغة للقراءة والاستماع

This document explains the implemented scope of the current application build, including the user-facing features, system architecture, backend services, database structure, content pipeline, and future extension path.

يشرح هذا المستند النطاق المنفذ فعلياً في النسخة الحالية من التطبيق، بما يشمل الخصائص الظاهرة للمستخدم، وبنية النظام، وخدمات الخلفية، وهيكل قاعدة البيانات، وآلية تجهيز المحتوى، ومسار التطوير المستقبلي.

110
Curated seed catalog entries configured in the backend.

2
Readable seed titles currently imported with full text.

2
Supported interface languages: English and Arabic.

2
Chapter audio voices available in the current app build.

PROJECT IDENTITY

Books Library

A Flutter-based digital library application connected to a Python FastAPI backend. The project combines book discovery, bilingual interface support, in-app reading, personal favorites, Firebase authentication, and chapter-level audio playback.

هوية المشروع

Books Library

تطبيق مكتبة رقمية مبني باستخدام Flutter ومرتبط بخلفية Python FastAPI. يجمع المشروع بين استكشاف الكتب، ودعم الواجهتين العربية والإنجليزية، والقراءة داخل التطبيق، والمفضلة الشخصية،

والمصادقة عبر Firebase، وتشغيل الصوت لكل فصل.

Generated on 12 April 2026 for academic project presentation and submission.

CONTENTS

English Report Structure

1. Project overview and objectives
2. Implemented features
3. User interface and screens
4. Technical architecture
5. Backend services and database
6. Content preparation and import pipeline
7. User workflow and data flow
8. Strengths and academic value
9. Current limitations
10. Future development roadmap

OVERVIEW

1. Project Overview and Objectives

Books Library was designed as a bilingual book application that presents English and Arabic titles in a modern, lightweight, and extendable format. The app does more than display catalog data: it allows users to browse, search, save favorites, create accounts, read available titles inside the application, and listen to chapter audio generated on demand.

The central objective of the project is to demonstrate a complete digital reading system rather than a static interface. For that reason, the work is divided into clear layers: a Flutter frontend, a FastAPI backend, a MySQL data store, content import scripts, media caching for audio, and Firebase-based authentication for user access.

Flutter + Dart **FastAPI + Python** **MySQL** **Firebase Auth** **Audio Playback** **English / Arabic UI**

> Current seed configuration in the backend defines 100 English titles and 10 Arabic titles, while two books are imported as fully readable content in the app build.

IMPLEMENTED SCOPE

2. Main Features Delivered

Curated bilingual home dashboard

The home screen loads catalog statistics, featured books, readable books, English books, Arabic books, and curated classics from backend data.

Dynamic search and filtering

Users can search by title or author and filter results by all books, readable books, English books, or Arabic books.

Personal favorites

Each book can be saved to favorites. The selection persists locally across sessions using `shared_preferences` on native platforms and `localStorage` on web.

Authentication and profile

Email and password sign-up, sign-in, sign-out, and profile display are implemented through Firebase Authentication.

Book details and reading state

Every title opens a detailed bottom sheet showing language, chapters, pages, description, and whether the book is fully readable or catalog-only.

In-app reading experience

Readable titles can be opened inside the app, where chapters and their page content are loaded and presented in a long-form reading layout.

Per-chapter audio generation

The chapter page can request MP3 narration from the backend, choose between Rachel and Adam voices, and control playback with play, pause, resume, and stop.

Runtime language switching

The entire interface can switch between English and Arabic using the in-app language toggle without rebuilding the project.

English report continues with interface, architecture, backend, and evaluation sections.

INTERFACE DESIGN

3. User Interface and Screen-Level Explanation

Main shell

A floating bottom navigation bar organizes the app into Favorites, Home, and Search. This creates a simple and memorable navigation structure.

Home page

Displays summary cards, featured titles, readable books, English and Arabic sections, and a curated grid, which helps users understand the library quickly.

Search page

Supports direct title and author lookup plus quick filters. This is useful when the catalog becomes larger.

Saved page

Shows favorite books only, creating a personalized shelf that survives app restarts.

Authentication page

Allows sign-up and sign-in through a compact email/password form connected to Firebase Authentication.

User page

Shows authenticated account data such as email and UID, plus logout functionality.

Book details sheet

Acts as the main decision point for each title by presenting metadata, chapter/page counts, reading availability, and a primary action button.

Reader and chapter pages

The reader shows full text across chapters, while the chapter page focuses on page-by-page reading and audio playback controls.

A notable UI detail is that the app does not depend on external cover image downloads for every title. Instead, it generates stylized cover artwork using gradients, monograms, badges, and typography. This keeps the interface visually consistent even when no real cover image is stored.

ARCHITECTURE

4. Technical Architecture

Flutter Frontend

- Material UI and custom theme
- Localization delegates for English and Arabic

Application Services

- `ApiService` for HTTP communication
- `FavoritesController` for local saved state

FastAPI Backend

- REST endpoints for books, chapters, pages, summaries, and audio

Data and Media

- MySQL tables for books and readable content
- Chapter audio cache table

- Pages, widgets, and lightweight state controllers

- `AuthService` for Firebase user actions

- Static media serving for generated MP3 files
- Schema initialization on startup

- Audio files stored under `/media/audio`

Layer	Role	Why it matters in this project
Presentation	Flutter pages and widgets	Provides a cross-platform interface for catalog browsing, reading, search, profile, and audio control.
State	ValueNotifier + InheritedNotifier	Keeps the architecture lightweight while still supporting live updates for locale changes and favorites.
Communication	HTTP + JSON	Separates UI from data services, making the app easier to extend and maintain.
Business logic	FastAPI routes and import scripts	Handles catalog queries, readable content access, audio generation, and source data preparation.
Persistence	MySQL and local storage	Supports both structured book data and personal client-side preferences.

The architecture intentionally separates UI, data access, authentication, and media generation into small, understandable modules.

BACKEND

5. Backend Services, API Design, and Database

The backend is implemented with FastAPI and starts by ensuring the required schema exists in MySQL. This keeps deployment simple: the app can initialize the database structure automatically when the server starts instead of relying on a separate migration system for this project stage.

The main API endpoints are:

- **`GET /books`**: returns the full catalog with language, featured state, chapter count, page count, and readability state.
- **`GET /books/summary`**: returns total books, readable books, English books, and Arabic books for the dashboard cards.
- **`GET /books/{book_id}/chapters`**: returns the chapter list for a selected book.
- **`GET /chapters/{chapter_id}/pages`**: returns page content for one chapter.
- **`GET /chapters/{chapter_id}/audio`**: generates or reuses a chapter MP3 file and returns a playable media URL.

Database Table	Purpose	Stored Data
`books`	Master catalog	Title, author, description, language, featured flag, sort order, created date
`chapters`	Readable structure	Book-to-chapter relationship, chapter number, chapter title
`pages`	Readable content units	Page number and text content for each chapter
`chapter_audio`	Audio cache index	Voice-specific generated audio path for each chapter

> The audio table uses a unique `(chapter_id, voice_name)` combination, which means the same chapter can cache different voices independently.

AUDIO PIPELINE

6. Audio Generation and Media Strategy

The audio service is one of the strongest technical parts of the project. When a user opens a chapter and requests narration, the backend loads chapter pages, builds a clean narration script, detects whether the content is mainly Arabic or English, and then tries to create MP3 audio.

The preferred provider is ElevenLabs when its SDK and API key are available. If that is not possible, the backend falls back to Google Text-to-Speech (`gTTS`). This fallback design improves reliability and demonstrates practical engineering, because the app can still function even when premium voice generation is unavailable.

Generated audio is stored under the backend media directory and reused in future requests. As a result, the first request may take a few seconds, but later playback is faster because the app reads from cache instead of regenerating the file.

Rachel voice **Adam voice** **Language detection** **Chunked text processing**

Cached MP3 output

The backend combines catalog APIs, media generation, caching, and static file serving in one coherent service layer.

CONTENT PIPELINE

7. Book Data Preparation and Import Workflow

The project does not rely only on manually typed catalog records. It includes dedicated Python scripts to prepare and import real readable content. This makes the system more realistic for a final year project because it demonstrates how raw source material can be transformed into structured application data.

English readable title import

`import_book.py` processes a clean JSON file for *Alice's Adventures in Wonderland*. It repairs misencoded text, normalizes headings, builds chapter entries, and writes the result into MySQL.

Arabic readable title import

`import_amwaj_akma.py` cleans raw Arabic page data, removes noise such as links and mixed-language artifacts, groups pages into sections, exports a generated JSON file, and imports the structured content into the database.

Seeded library catalog

`catalog_seed.py` inserts the full curated library list, applies featured ordering, resets old content when needed, and re-imports the readable titles so the app starts from a known academic demo dataset.

Catalog strategy

The app intentionally supports both catalog-only books and fully readable books. This allows the project to scale the library now while still expanding full-text content progressively later.

FLOW

8. User Flow and Runtime Data Flow

Step	User action	System response
1	User opens the app	Flutter initializes Firebase, loads favorites from storage, and shows the main shell.
2	User enters Home or Search	`ApiService` requests catalog data and renders books using model classes.
3	User opens a book	The details sheet displays metadata and decides whether reading is available.
4	User reads a title	The app loads chapters and pages and builds a long-form reading view.
5	User requests audio	The backend generates or reuses chapter audio, returns a URL, and the app streams it with `audioplayers`.

Step	User action	System response
6	User favorites a book or changes language	State updates immediately in the UI through lightweight notifiers.

EVALUATION

9. Strengths, Academic Value, and Current Limitations

Strengths

- Clear separation between frontend, backend, storage, and media generation
- Strong bilingual support at the UI and content level
- Functional reading and audio features instead of a catalog-only demo
- Practical content import workflow for future expansion

Current limitations

- Only two seeded titles currently include full readable content
- Favorites are local to the device and not cloud-synced per account
- Authentication is limited to email/password and basic profile data
- No admin dashboard or recommendation engine has been added yet

A strong final-year project is not only about features but also about system structure, extensibility, and demonstrable engineering decisions. This project has all three.

FUTURE SCOPE

10. Recommended Next Development Steps

Expand readable content

Import additional English and Arabic titles so a larger percentage of the catalog becomes fully readable inside the app.

Cloud-synced user library

Store favorites, reading progress, and voice preferences per account so users keep their experience across devices.

Offline reading mode

Cache chapters and audio locally to support limited connectivity and improve performance.

Admin content dashboard

Create a management panel to add books, import files, edit metadata, and review generated media.

Advanced recommendations

Add category-based discovery, recently read titles, and behavior-driven suggestions.

Stronger production readiness

Introduce secure environment management, deployment configuration, analytics, testing expansion, and richer error monitoring.

CONCLUSION

11. English Conclusion

Books Library is a complete and credible final year project because it demonstrates both product thinking and real engineering implementation. The application is not limited to a static user interface; it includes structured data management, authentication, localized user experience, real reading flows, audio generation, and a backend architecture that can continue growing.

In academic terms, the project shows the full lifecycle of a software product: content preparation, database design, API construction, frontend development, persistent state handling, user account integration, and feature delivery in a usable interface. This makes the project suitable not only for demonstration, but also for technical discussion during evaluation.

بداية التقرير العربي

التقرير العربي

تبدأ الصفحات التالية بعرض نفس المشروع باللغة العربية، مع التركيز على الفكرة العامة، والخصائص المنفذة، والبنية التقنية، وآلية تجهيز الكتب، والقيمة العلمية للمشروع، ومجالات التطوير المستقبلية.

Arabic section starts on the next page.

نظرة عامة

1. مقدمة عن المشروع وأهدافه

تم تطوير تطبيق **Books Library** كمكتبة رقمية ثنائية اللغة تعرض كتباً باللغة الإنجليزية والعربية داخل واجهة حديثة وقابلة للتوسع. المشروع لا يقتصر على عرض قائمة كتب فقط، بل يوفر استكشافاً للمحتوى، والبحث، والمفضلة، وإنشاء الحسابات، والقراءة داخل التطبيق، وتشغيل الصوت لكل فصل عند الطلب.

الهدف الرئيسي من المشروع هو تقديم نظام قراءة رقمي متكامل يوضح التكامل بين الواجهة الأمامية والخلفية وقاعدة البيانات وخدمات الوسائط. لذلك تم تقسيم النظام إلى طبقات واضحة: واجهة Flutter، وخادم FastAPI، وقاعدة بيانات MySQL، وسكربتات استيراد للمحتوى، وتخزين مؤقت للصوت، ومصادقة للمستخدمين عبر Firebase.

واجهة Flutter | خادم FastAPI | قاعدة MySQL | Firebase Auth | صوت الفصول

العربية والإنجليزية

> في التهيئة الحالية للمكتبة توجد 100 رواية أو كتاب باللغة الإنجليزية و10 كتب باللغة العربية ضمن الفهرس، بينما تم استيراد كتابين فقط بشكل كامل للقراءة داخل التطبيق.

النطاق المنفذ

2. الخصائص التي تم إنجازها

الواجهة الرئيسية للمكتبة

تعرض الصفحة الرئيسية إحصاءات المكتبة والكتب المميزة والكتب الجاهزة للقراءة وأقسام الإنجليزية والعربية ومجموعة مختارة من العناوين.

البحث والتصفية

يمكن للمستخدم البحث باسم الكتاب أو المؤلف، مع تصفية النتائج حسب جميع الكتب أو الكتب المتاحة للقراءة أو الكتب الإنجليزية أو العربية.

المفضلة الشخصية

يمكن حفظ أي كتاب ضمن المفضلة، ويتم الاحتفاظ بهذه البيانات محلياً حتى بعد إغلاق التطبيق وإعادة تشغيله.

إنشاء الحساب وتسجيل الدخول

يوفر التطبيق التسجيل وإنشاء الحساب والخروج باستخدام البريد الإلكتروني وكلمة المرور من خلال Firebase Authentication.

صفحة تفاصيل الكتاب

توضح حالة الكتاب إن كان قابلاً للقراءة أو مجرد عنوان ضمن الفهرس، وتعرض اللغة وعدد الفصول والصفحات والوصف.

القراءة داخل التطبيق

الكتب المتاحة كاملة يمكن قراءتها داخل التطبيق بطريقة منظمة حسب الفصول والمحتوى النصي لكل صفحة.

الصوت لكل فصل

توفر صفحة الفصل إمكانية توليد ملف MP3 وتشغيله مع اختيار الصوت والتحكم بالإيقاف المؤقت والمتابعة والإيقاف.

تبديل اللغة مباشرة

يمكن الانتقال بين العربية والإنجليزية من داخل التطبيق بشكل فوري دون الحاجة إلى بناء نسخة جديدة.

يتابع التقرير العربي بشرح الواجهات والبنية التقنية وقاعدة البيانات وآلية العمل.

شرح الواجهات

3. شرح الشاشات وتجربة الاستخدام

الهيكل الرئيسي
يعتمد التطبيق على شريط تنقل سفلي يضم المفضلة والرئيسية والبحث، مما يجعل الوصول إلى الوظائف الأساسية واضحاً وسريعاً.

الصفحة الرئيسية
تُظهر بطاقات إحصائية وأقساماً متعددة للمكتبة، وهو ما يمنح المستخدم تصوراً سريعاً عن حجم المحتوى وطبيعته.

صفحة البحث
تسمح بالعثور على الكتب مباشرة عند كبر حجم الفهرس، مع أدوات تصفية واضحة وسهلة.

صفحة المفضلة
تجمع الكتب المحفوظة من المستخدم في مكان واحد لتكوين رف شخصي داخل التطبيق.

صفحة المصادقة
توفر نموذجاً بسيطاً لتسجيل الدخول أو إنشاء حساب باستخدام البريد الإلكتروني وكلمة المرور.

صفحة المستخدم
تعرض معلومات الحساب مثل البريد الإلكتروني والمعرف وتسمح بتسجيل الخروج.

نافذة تفاصيل الكتاب
تمثل نقطة القرار الأساسية، حيث تعرض حالة الكتاب ومعلوماته الرئيسية وزر القراءة داخل التطبيق.

صفحتا القراءة والفصل
تعرض صفحة القراءة النص الكامل على مستوى الفصول، بينما تركز صفحة الفصل على النص الجزئي والصوت والتحكم بالمشغل.

ومن الجوانب الجمالية المهمة أن التطبيق لا يعتمد بالكامل على صور أغلفة جاهزة، بل ينشئ أغلفة بصرية موحدة باستخدام التدرجات اللونية والأحرف المختصرة والشارات، مما يحافظ على هوية تصميمية ثابتة حتى عندما لا تتوفر صورة حقيقية للكتاب.

البنية التقنية

4. بنية النظام

الواجهة الأمامية
- Flutter وMaterial UI
- ثيم موحد وألوان خاصة بالمشروع
- دعم لغتين داخل التطبيق

خدمات التطبيق
- ApiService لطلبات HTTP
- FavoritesController لإدارة المفضلة
- AuthService للتعامل مع Firebase

الخادم الخلفي
- واجهات REST للكتب والفصول والصفحات والصوت
- خدمة ملفات ثابتة للصوت
- تهيئة الجداول عند بدء التشغيل

البيانات والوسائط
- MySQL لتخزين المحتوى والفهرس
- جدول خاص لتخزين كاش الصوت
- ملفات MP3 ضمن مجلد الوسائط

الطبقة	الدور	الأهمية داخل المشروع
العرض	صفحات وودجت Flutter	تقديم واجهة متعددة المنصات لعرض الكتب والقراءة والبحث والصوت والحساب الشخصي.
الحالة	ValueNotifier وInheritedNotifier	حل خفيف وواضح لإدارة اللغة والمفضلة دون الحاجة إلى مكتبات إدارة حالة كبيرة.
الاتصال	HTTP وJSON	فصل الواجهة عن البيانات والخدمات الخلفية لتسهيل الصيانة والتوسع.
المنطق التشغيلي	FastAPI وسكربتات الاستيراد	تنفيذ الاستعلامات وتجهيز المحتوى وتوليد الصوت وتقديمه للتطبيق.
التخزين	MySQL والتخزين المحلي	حفظ بيانات الكتب من جهة، وبيانات المستخدم المحلية مثل المفضلة من جهة أخرى.

اعتماد هذه البنية يجعل المشروع واضحاً أثناء المناقشة الأكاديمية وسهل التوسعة لاحقاً.

الخلفية وقاعدة البيانات

5. الخادم الخلفي والواجهات البرمجية وقاعدة البيانات

تم بناء الخادم الخلفي باستخدام FastAPI، ويقوم عند التشغيل بالتأكد من وجود الجداول الأساسية داخل قاعدة بيانات MySQL. هذا الأسلوب يجعل تشغيل المشروع أسهل في بيئة العرض أو المناقشة، لأن الخادم يستطيع تجهيز البنية المطلوبة تلقائياً.

أهم الواجهات البرمجية المنفذة هي:

- **`GET /books`** لإرجاع جميع الكتب مع اللغة والحالة وعدد الفصول والصفحات.
- **`GET /books/summary`** لإرجاع إحصاءات الصفحة الرئيسية.
- **`GET /books/{book_id}/chapters`** لإرجاع فصول كتاب محدد.
- **`GET /chapters/{chapter_id}/pages`** لإرجاع نص الصفحات التابعة لفصل معين.
- **`GET /chapters/{chapter_id}/audio`** لتوليد أو إعادة استخدام ملف الصوت وإرجاع رابط التشغيل.

اسم الجدول	الغرض	نوع البيانات المخزنة
`books`	الفهرس الرئيسي	اسم الكتاب والمؤلف والوصف واللغة والترتيب والحالة المميزة
`chapters`	هيكلية الكتاب	ربط الكتاب بالفصول وأرقامها وعناوينها
`pages`	النص المقروء	رقم الصفحة والمحتوى النصي لكل فصل
`chapter_audio`	فهرس كاش الصوت	مسار ملف الصوت لكل فصل ولكل صوت مختار

> يستخدم جدول الصوت مفتاحاً فريداً يربط الفصل باسم الصوت، لذلك يمكن حفظ أكثر من نسخة صوتية للفصل نفسه مثل Rachel وAdam.

نظام الصوت

6. آلية توليد الصوت وتخزينه

عند طلب المستخدم الاستماع إلى فصل معين، يجمع الخادم النص الكامل للفصل، ثم يحدد إن كانت اللغة الغالبة عربية أو إنجليزية، وبعد ذلك يبدأ إنشاء ملف MP3.

يتم استخدام ElevenLabs أولاً عند توفر المفتاح والمكتبة المطلوبة، وإذا تعذر ذلك يتم الانتقال تلقائياً إلى Google Text-to-Speech. هذه الآلية مهمة لأنها تضيف مرونة عالية للمشروع وتمنع توقف الميزة عند غياب مزود واحد فقط.

بعد إنشاء الصوت يتم حفظه داخل مجلد الوسائط وتسجيله في قاعدة البيانات ليعاد استخدامه لاحقاً. لذلك قد تكون أول عملية أبطأ قليلاً، بينما تصبح المرات اللاحقة أسرع بسبب وجود الكاش.

Rachel | Adam | كشف اللغة | تقسيم النص الطويل | تخزين MP3

ميزة الصوت تعد من أهم العناصر التي ترفع قيمة المشروع من فهرس كتب إلى منصة قراءة واستماع فعلية.

تجهيز المحتوى

7. آلية استيراد الكتب وتجهيز البيانات

لا يعتمد المشروع على إدخال يدوي بسيط للكتب فقط، بل يحتوي على سكربتات Python مخصصة لتحويل ملفات المصدر إلى بيانات منظمة يمكن للتطبيق قراءتها. هذا يعطي المشروع بعداً عملياً مهماً، لأنه يوضح كيف يمكن الانتقال من ملف خام إلى نظام قراءة كامل.

استيراد الكتاب الإنجليزي

يقوم سكربت `import_book.py` بمعالجة ملف JSON لكتاب *Alice's Adventures in Wonderland*، وإصلاح ترميز النص وتنظيم العناوين ثم كتابة الفصول داخل قاعدة البيانات.

استيراد الكتاب العربي

يقوم سكربت `import_amwaj_akma.py` بتنظيف الصفحات العربية الخام، وحذف الروابط والضجيج والآثار المختلطة، ثم تقسيم المحتوى إلى أقسام وتصديره أيضاً بصيغة JSON منظمة قبل إدخاله إلى القاعدة.

بذر المكتبة الكاملة

يستخدم `catalog_seed.py` لإدخال قائمة الكتب الكاملة مع الترتيب والتمييز وإعادة تحميل المحتوى المقروء، حتى تبدأ النسخة التجريبية للمشروع ببيانات واضحة ومضبوطة.

استراتيجية الفهرس

يدعم التطبيق حالتين: كتب متاحة كاملة للقراءة، وكتب موجودة في الفهرس فقط. هذا القرار يجعل المشروع قابلاً للتوسع التدريجي دون الحاجة إلى تجهيز النص الكامل لكل كتاب منذ البداية.

تدفق العمل

8. رحلة المستخدم وتدفق البيانات

الخطوة	فعل المستخدم	استجابة النظام
1	فتح التطبيق	يتم تهيئة Firebase وتحميل المفضلة وعرض الهيكل الرئيسي للتطبيق.
2	الدخول إلى الرئيسية أو البحث	تستدعي الواجهة بيانات الكتب من الخادم عبر `ApiService` ثم تعرضها باستخدام النماذج البرمجية.
3	فتح كتاب معين	تظهر نافذة التفاصيل وتحدد هل الكتاب متاح للقراءة أم أنه ضمن الفهرس فقط.
4	بدء القراءة	يتم تحميل الفصول والصفحات وبناء واجهة قراءة طويلة داخل التطبيق.
5	طلب الصوت	يقوم الخادم بتوليد الصوت أو استرجاعه من الكاش ثم يعيد رابط الملف ليتم تشغيله داخل التطبيق.
6	تغيير اللغة أو إضافة كتاب للمفضلة	تتحدث الواجهة مباشرة بفضل آلية إدارة الحالة الخفيفة المستخدمة في المشروع.

التقييم

9. نقاط القوة والقيود الحالية

نقاط القوة

- فصل واضح بين الواجهة والخلفية والبيانات والوسائط
- دعم حقيقي للغتين العربية والإنجليزية
- وجود قراءة وصوت فعليين وليس مجرد عرض فهرس
- إمكانية توسيع المحتوى عبر سكربتات الاستيراد

القيود الحالية

- عدد الكتب الكاملة للقراءة ما زال محدوداً في النسخة الحالية
- المفضلة محفوظة محلياً وليست مرتبطة بالسحابة
- المصادقة تعتمد على البريد الإلكتروني وكلمة المرور فقط
- لا توجد لوحة إدارة أو نظام توصيات حتى الآن

من الناحية الأكاديمية، يبرهن المشروع على فهم جيد للتكامل بين الواجهة والخلفية وإدارة المحتوى وتجربة المستخدم.

التطوير المستقبلي

10. مقترحات التطوير اللاحق

زيادة عدد الكتب الكاملة

إضافة كتب عربية وإنجليزية جديدة قابلة للقراءة بحيث تصبح نسبة المحتوى الكامل أعلى داخل التطبيق.

مزامنة المكتبة الشخصية

حفظ المفضلة وتقدم القراءة وتفضيلات الصوت على مستوى الحساب بدلاً من التخزين المحلي فقط.

القراءة دون اتصال

إتاحة تخزين النصوص والصوت محلياً لدعم الاستخدام في بيئات الاتصال المحدود.

لوحة إدارة للمحتوى

إنشاء واجهة تحكم لإضافة الكتب واستيراد الملفات وتعديل البيانات الوصفية ومراجعة الوسائط الناتجة.

اقتراحات ذكية للكتب

إضافة آليات توصية حسب اللغة أو الاهتمام أو سجل القراءة السابق.

جاهزية إنتاجية أعلى

تحسين إدارة الأسرار، وإضافة تحليلات، وتوسيع الاختبارات، وتعزيز المراقبة والتعامل مع الأخطاء.

الخاتمة

11. الخاتمة العربية

يُعد تطبيق **Books Library** مشروع تخرج قوياً لأنه يجمع بين التفكير المنتج والتطبيق الهندسي الحقيقي. فالمشروع لا يقدم واجهة شكلية فقط، بل يوفر منظومة متكاملة تشمل إدارة المحتوى، وقاعدة البيانات، والمصادقة، وتجربة استخدام ثنائية اللغة، والقراءة داخل التطبيق، وتوليد الصوت عند الطلب.

ومن الناحية العلمية، يوضح المشروع دورة بناء منتج برمجي كاملة: تجهيز البيانات، وتصميم قاعدة البيانات، وبناء الواجهات البرمجية، وتطوير الواجهة الأمامية، وإدارة الحالة، وربط الحسابات، وتقديم خصائص نهائية قابلة للعرض والمناقشة. لذلك فهو مناسب جداً للعرض الأكاديمي ولمناقشة الجوانب التقنية والعملية في الوقت نفسه.

End of bilingual report.